

Manual Técnico del Sistema Integrado de Predicción y Monitoreo de Sequías Hidrometeorológicas (SIPREH)

**Grupo de Investigación en Ingeniería de los Recursos Hídricos —
GIREH**

Facultad de Ingeniería, Universidad Nacional de Colombia

24 de junio de 2026

Capítulo 1. Introducción	4
1.1. Propósito del sistema	4
1.2. Objetivos principales del sistema	4
1.3. Público objetivo	5
Capítulo 2. Previsualización	7
2.1. Descripción del sistema	7
2.2. Funcionalidades principales.....	7
2.2.1. Gráficas 1D	7
2.2.2. Gráficas 2D	8
2.2.3. Exportación de datos	8
2.2.4. Administración	8
Capítulo 3. Tecnologías	9
3.1. Frontend.....	9
3.2. Backend	10
3.3. Base de datos	11
Capítulo 4. Arquitectura del Sistema	12
4.1. Descripción y estructura del SIPREH.....	12
4.2. Relaciones geográficas entre celdas y zonas	13
Capítulo 5. Principales Funcionalidades del Sistema	14
5.1. Datos históricos	14
5.2. Predicción actual.....	15
5.3. Predicción histórica	16
5.4. Administración.....	16
Capítulo 6. Funcionamiento del Sistema	20
6.1. Flujograma de acceso del usuario.....	20
6.2. Flujograma de acceso del administrador	21
Capítulo 7. Estructura del Proyecto	22

7.1. Estructura del frontend.....	22
7.2. Estructura del backend.....	23
Capítulo 8. Despliegues	25
8.1. Despliegue frontend.....	25
8.2. Despliegue backend.....	26
8.3. Almacenamiento y base de datos	28

Capítulo 1 Introducción

El Sistema Integrado de Predicción y Monitoreo de Sequías Hidrometeorológicas (SIPREH) es una plataforma web diseñada para el análisis continuo, el monitoreo histórico y la predicción probabilística de condiciones de sequía sobre la ciudad de Bogotá D.C. y las siete principales cuencas hidrográficas que abastecen su sistema de acueducto: La Regadera, Chisacá, Sisga, Chuza, Neusa, Tominé y San Rafael.

El sistema integra datos climáticos derivados de satélite y productos de reanálisis con salidas de modelos hidrometeorológicos numéricos, presentando los resultados a través de una interfaz geoespacial interactiva orientada a investigadores, gestores de recursos hídricos y tomadores de decisiones públicas.

SIPREH se enmarca en el proyecto *Sistema de Análisis y Alerta de Sequías Meteorológicas e Hidrológicas para Bogotá y para las cuencas hidrográficas y embalses que abastecen de agua a la ciudad*, financiado por la Agencia Distrital para la Educación Superior, la Ciencia y la Tecnología (ATENEA) y desarrollado por el Grupo de Investigación en Ingeniería de los Recursos Hídricos (GIREH) de la Facultad de Ingeniería de la Universidad Nacional de Colombia.

1.1. Propósito del sistema

SIPREH fue concebido para responder a la necesidad institucional de contar con una herramienta integrada que permita monitorear, analizar y predecir condiciones de sequía sobre el área metropolitana de Bogotá y sus embalses de abastecimiento. El sistema cubre un dominio geográfico aproximado de 4.0°N–5.5°N, 73.5°W–74.5°W.

Sus propósitos centrales son:

- ✓ Monitorear el estado de sequía sobre Bogotá y sus cuencas de abastecimiento mediante índices meteorológicos e hidrológicos calculados sobre múltiples fuentes de datos.
- ✓ Analizar series históricas de índices de sequía con cobertura espacial continua y resolución temporal mensual.
- ✓ Visualizar predicciones probabilísticas de hasta doce meses de horizonte con cuantificación explícita de incertidumbre.
- ✓ Facilitar la comparación entre distintas versiones de predicción emitidas en diferentes fechas.
- ✓ Habilitar la exportación de datos para análisis externos e integración con otros sistemas de gestión.
- ✓ Proveer un módulo de administración para la carga, configuración y sincronización de archivos de datos.

1.2. Objetivos principales del sistema

El desarrollo de SIPREH persigue los siguientes objetivos técnicos y operativos:

- 1. Integración multifuente:** Consolidar en una única plataforma datos de reanálisis (ERA5, ERA5-Land),

precipitación satelital (IMERG, CHIRPS) y registros observados de 29 estaciones hidrológicas del IDEAM.

2. **Cómputo y visualización de índices:** Calcular y desplegar diez índices de sequía —cinco meteorológicos (SPI, SPEI, RAI, EDDI, PDSI) y cinco hidrológicos (SDI, SRI, MFI, DDI, HDI)— a múltiples escalas temporales (1, 3, 6 y 12 meses).
3. **Predicción con cuantificación de incertidumbre:** Incorporar predicciones de conjunto con bandas de incertidumbre (Q1, Q3, IQR) para horizontes de hasta 12 meses.
4. **Interactividad geoespacial:** Proveer un mapa interactivo con drill-down a nivel de celda y polígonos de cuencas, integrado con gráficas de series de tiempo que responden al clic del usuario sin recarga de página.
5. **Eficiencia computacional:** Garantizar latencias de consulta compatibles con la exploración interactiva mediante caché de dos niveles (Redis e in-memory) y consultas analíticas sobre archivos Parquet con DuckDB.
6. **Trazabilidad de predicciones:** Archivar versiones históricas de predicción identificadas por fecha de emisión (*issued_at*), permitiendo la comparación retrospectiva de distintas corridas.
7. **Resumen asistido por IA:** Generar automáticamente interpretaciones en lenguaje natural del estado de sequía predicho, reduciendo la barrera de acceso para usuarios no especializados.
8. **Administración operativa:** Facilitar la gestión de archivos, conjuntos de datos, usuarios y sincronización con almacenamiento en la nube mediante un módulo de administración web.

1.3. Público objetivo

SIPREH está diseñado para tres perfiles de usuario diferenciados, cada uno con necesidades y niveles de acceso distintos:

Definición 1.1

Investigador o científico. Usuario con formación técnica en hidrología, climatología o áreas afines. Explora series históricas de índices, compara fuentes de datos y exporta información para análisis cuantitativos. Requiere acceso a todos los índices disponibles, múltiples escalas temporales y capacidades de exportación en CSV y PNG.

Definición 1.2

Gestor de recursos hídricos. Profesional operativo de entidades como la Empresa de Acueducto y Alcantarillado de Bogotá (EAAB), la Corporación Autónoma Regional de Cundinamarca (CAR) o el Instituto Distrital de Gestión de Riesgos y Cambio Climático (IDIGER). Enfocado en el seguimiento de la condición actual de sequía y las alertas tempranas a corto plazo. Su interés principal reside en la visualización clara del estado actual y la predicción de los próximos meses.

Definición 1.3

Administrador del sistema. Usuario técnico responsable de la carga de archivos de datos, la configuración de conjuntos de datos, la gestión de cuentas de usuario y la sincronización con el almacenamiento en la nube. Accede al sistema mediante credenciales con privilegios de administrador y opera el módulo de administración integrado en la plataforma.

 Observación

Las secciones de visualización de datos históricos y de predicción son accesibles para todos los usuarios autenticados. El módulo de administración es exclusivo para usuarios con rol de administrador.

Capítulo 2 Previsualización

Este capítulo presenta una visión general de la interfaz de SIPREH y sus funcionalidades desde la perspectiva del usuario.

2.1. Descripción del sistema

SIPREH es una aplicación web de página única (SPA) organizada en torno a un mapa interactivo central y un panel lateral contextual. La navegación principal se divide en tres secciones:

1. **Datos históricos:** visualización de índices de sequía sobre el mapa y en gráficas de series de tiempo.
2. **Predicción actual:** mapa del pronóstico probabilístico vigente con horizonte de 12 meses.
3. **Predicción histórica:** comparación entre versiones de predicción emitidas en distintas fechas.

Los usuarios con rol de administrador también pueden acceder al panel de administración para gestionar archivos, conjuntos de datos, usuarios y sincronización con la nube.

Observación

La interfaz está optimizada para navegadores de escritorio con resolución mínima de 1366 × 768 píxeles.

2.2. Funcionalidades principales

2.2.1. Gráficas 1D

Las gráficas 1D muestran la serie de tiempo de un índice de sequía para una celda o cuenca seleccionada. Se activan al hacer clic sobre el mapa interactivo.

Capacidades:

- ✓ Zoom y desplazamiento sobre el eje temporal.
- ✓ Código de color por categoría de severidad.
- ✓ Exportación de la gráfica como imagen PNG.
- ✓ Descarga de los datos en formato JSON.

Observación

Las gráficas 1D utilizan la librería `uPlot` con el algoritmo de diezmado LTTB (*Largest-Triangle-Three-Buckets*), que preserva la forma visual de la serie al reducir el número de puntos renderizados en registros extensos.

2.2.2. Gráficas 2D

Las gráficas 2D corresponden al campo espacial del índice seleccionado para una fecha determinada, visualizado como un mapa de color continuo. El mapa se actualiza al cambiar la fecha, el índice o la escala en el panel lateral.

Capacidades:

- ✓ Campo espacial del índice con paleta de colores estándar de sequía.
- ✓ Superposición de los siete polígonos de cuencas hidrográficas.
- ✓ Marcadores de estaciones del IDEAM con acceso a sus series de tiempo.
- ✓ Tres niveles de resolución según el nivel de zoom.
- ✓ Exportación del mapa como imagen PNG.

2.2.3. Exportación de datos

El sistema permite exportar los datos visualizados en dos formatos, generados directamente en el navegador:

- ✓ **PNG:** captura de alta resolución de cualquier panel de mapa o gráfica. Se descarga con un encabezado institucional, tipo de análisis, leyenda y metadatos.
- ✓ **JSON:** datos de la consulta activa (serie de tiempo 1D o campo espacial 2D), incluyendo valores, metadatos de variable, fuente, resolución y sistema de referencia (WGS84 EPSG:4326).

Ejemplo 2.1

El JSON de una consulta 1D contiene columnas `Tiempo` y `Valor`. El JSON de una consulta 2D contiene `Cell_id` y `Valor`. Para cuencas, incluye además `Cuenca_DN`, `Nombre_Cuenca` y `Categoria`.

2.2.4. Administración

El panel de administración es exclusivo para usuarios con rol de administrador. Desde él se puede:

- ✓ Cargar archivos Parquet de datos al sistema.
- ✓ Configurar y activar conjuntos de datos.
- ✓ Sincronizar el almacenamiento local con Cloudflare R2.
- ✓ Gestionar usuarios (crear, modificar, desactivar cuentas).

3.1. Frontend

Tecnología	Versión	Función
Next.js	16	Framework React (App Router). En producción se exporta como sitio estático para GitHub Pages.
React	18	Librería de componentes reactivos para la interfaz de usuario.
TypeScript	5	Superconjunto tipado de JavaScript; mejora la mantenibilidad del código.
Leaflet.js	≥ 1.9	Mapa interactivo: grilla de índices, polígonos de cuencas, marcadores.
uPlot	≥ 1.6	Gráficas 1D de alto rendimiento con zoom, pan y diezmado LTTB.
Canvas 2D	–	API nativa del navegador para gráficas de predicción con bandas IQR.

3.2. Backend

Tecnología	Versión	Función
Python	3.11+	Lenguaje base; integración de librerías científicas y de análisis.
FastAPI	≥ 0.110	Framework REST asíncrono; genera documentación Swagger/ReDoc automáticamente.
Uvicorn	–	Servidor ASGI que ejecuta FastAPI con soporte de peticiones concurrentes.
DuckDB	≥ 0.10	Motor analítico embebido; consulta Parquet directamente, 10–100× más rápido que Pandas.
PyArrow	≥ 15	Lectura/escritura de Parquet; extrae metadatos al cargar archivos.
Redis	–	Caché de resultados con TTL configurable (900 s); $\sim 50\times$ speedup.
PyJWT	–	Generación y validación de tokens JWT para control de acceso.
boto3	–	SDK S3 de AWS para comunicación con Cloudflare R2.
Groq API	–	Resúmenes en lenguaje natural vía modelo Llama 3.1-8b-instant.

Observación

DuckDB opera en modo *in-process*: no requiere servidor separado y se embebe directamente en el proceso Python, eliminando la latencia de red entre aplicación y motor de consultas.

3.3.

Base de datos

Tecnología	Entorno	Función
SQLite	Desarrollo	BD relacional embebida. Metadatos, datasets, usuarios y tokens. Sin servidor.
PostgreSQL	Producción	BD relacional de servidor completo para mayor concurrencia y robustez.
Cloudflare R2	Producción	Almacenamiento de objetos (compatible S3). Aloja los archivos Parquet. Sin costos de egreso.
Apache Parquet	Formato	Formato columnar de alta compresión. Esquema: {date, lat, lon, variable, value, scale, source}.

Ejemplo 3.1

En el dominio de estudio a resolución CHIRPS (0,05°), el archivo histórico contiene 294 celdas × 12 meses × 30 años = 105 840 filas por índice y escala. Con múltiples índices y escalas, el archivo consolidado puede superar los 50 millones de filas, donde DuckDB sobre Parquet ofrece una ventaja de rendimiento decisiva.

Capítulo 4 Arquitectura del Sistema

SIPREH sigue una arquitectura de tres capas débilmente acopladas: (i) almacenamiento en la nube, (ii) backend de procesamiento y API REST, y (iii) frontend web reactivo. Cada capa se comunica exclusivamente a través de la API HTTP versionada bajo `/api/v1`.

4.1. Descripción y estructura del SIPREH

La Figura 4.1 muestra el flujo de datos desde las fuentes originales hasta el usuario final.

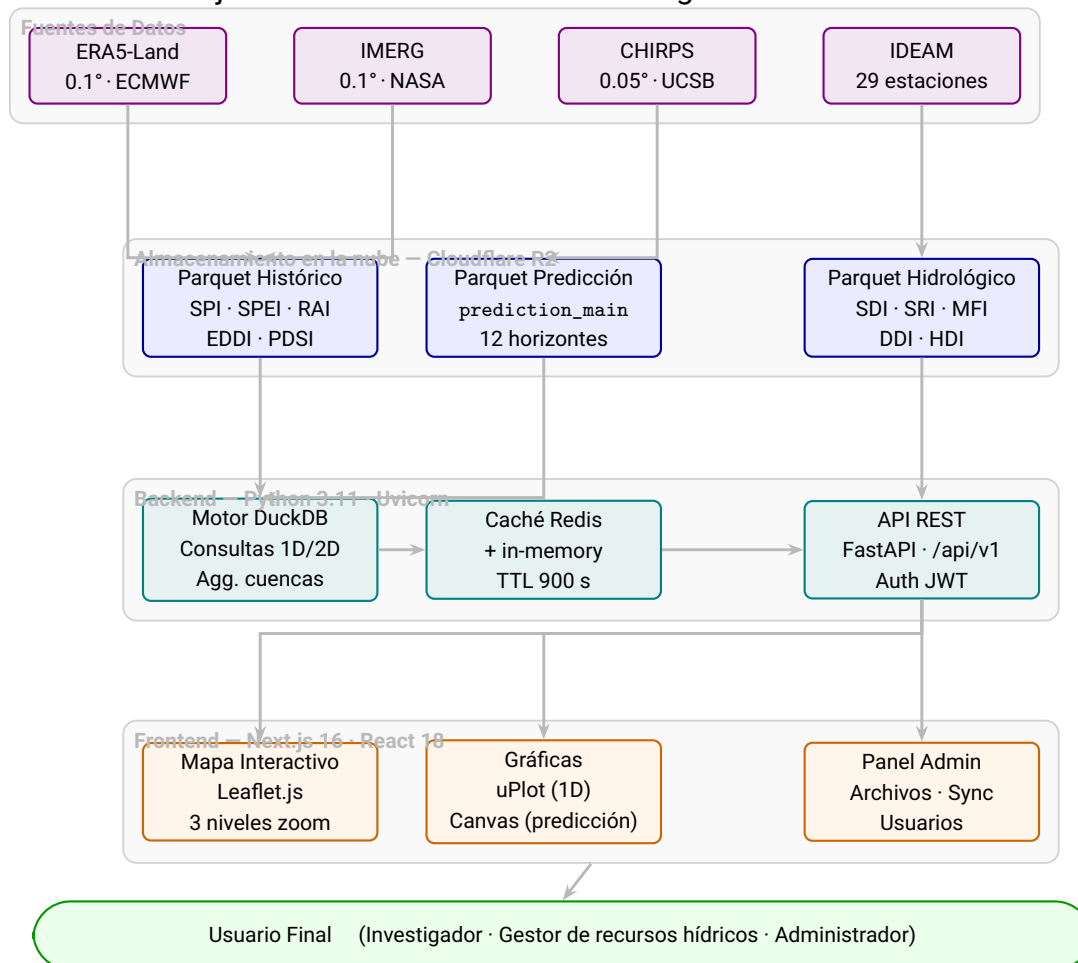


Figura 4.1. Arquitectura en capas del SIPREH. Los datos fluyen verticalmente desde las fuentes hasta el usuario final a través del almacenamiento en la nube, el backend y el frontend.

El flujo de datos dentro del sistema sigue estos pasos:

- 1. Ingesta:** Los datos de ERA5-Land, IMERG y CHIRPS se procesan externamente por el pipeline y se almacenan como archivos Parquet en el bucket R2. Los datos del IDEAM se almacenan como Parquet hidrológico.
- 2. Descarga y caché de disco:** En la primera petición sobre un archivo, el backend lo descarga desde R2 y lo almacena localmente (hasta 10 GB). Las peticiones siguientes leen desde disco.
- 3. Procesamiento:** DuckDB ejecuta consultas SQL sobre los archivos Parquet. Soporta tres patrones: 1D (serie de tiempo por celda), 2D (campo espacial por fecha) y agregación por cuenca (promedio ponderado por área).

4. **Caché de resultados:** Los resultados se almacenan en Redis con TTL de 900 s. Los accesos en caché devuelven respuestas en tiempos de un solo dígito de milisegundos ($\sim 50\times$ más rápido).
5. **API REST:** FastAPI expone los resultados bajo `/api/v1` con routers para autenticación, histórico, predicción, datos hidrológicos, administración y dashboard.
6. **Visualización:** Next.js consume la API y renderiza el mapa (Leaflet.js), las gráficas 1D (uPlot) y las gráficas de predicción (Canvas 2D).

4.2. Relaciones geográficas entre celdas y zonas

Muchas consultas de SIPREH no se piden por celda, sino por **zona**: una cuenca, un municipio o un perímetro urbano. Para responderlas, el sistema necesita saber qué celdas caen dentro de cada zona y en qué proporción. Esa correspondencia es *metadata geográfica* y, por eso, vive en la base de datos relacional, no en la capa de datos científicos.

El modelo es una relación muchos-a-muchos normalizada, repartida en tres tablas:

Tabla	Qué guarda
<code>grid_cells</code>	Las celdas del grid, identificadas por su fuente/resolución (ERA5, ERA5-Land, IMERG o CHIRPS) y su <code>cell_id</code> (<code>lon_lat</code>).
<code>spatial_zones</code>	Las entidades espaciales: cuenca, municipio o perímetro, cada una con su tipo, su identificador y su nombre.
<code>zone_cell_relations</code>	La unión entre ambas. Cada fila enlaza una zona con una celda y guarda el área de intersección (<code>area_m2</code>) como peso de esa unión.

Definición 4.1

Agregación por zona: el valor de una zona es el promedio de sus celdas ponderado por el área que cada celda aporta a la zona:

$$\text{valor}_{\text{zona}} = \frac{\sum_c \text{valor}_c \times \text{área}_c}{\sum_c \text{área}_c}$$

Como las áreas de intersección están precalculadas y almacenadas en `zone_cell_relations`, la agregación en tiempo de consulta se reduce a esta suma ponderada, sin recomputar geometría.

Observación

Guardar las relaciones en tablas (en lugar de calcularlas al vuelo en cada petición) tiene dos ventajas: las consultas por zona son inmediatas y el cruce geométrico —costoso— se hace una sola vez. Añadir un nuevo conjunto de zonas se reduce a poblar `spatial_zones` y `zone_cell_relations` mediante el sembrado correspondiente, de forma idempotente y sin tocar la interfaz.

Capítulo 5 Principales Funcionalidades del Sistema

5.1. Datos históricos

La sección de datos históricos permite consultar el estado de sequía pasado mediante índices calculados sobre múltiples fuentes de datos. El usuario selecciona la fuente de datos, el índice, la escala temporal y la fecha; el sistema actualiza el mapa espacial y, al hacer clic en una celda, muestra la serie de tiempo correspondiente.

? Índices meteorológicos

Índice	Escalas	Descripción
SPI	1, 3, 6, 12 m	Anomalía de precipitación normalizada respecto a la distribución histórica.
SPEI	1, 3, 6, 12 m	Extiende el SPI incorporando la evapotranspiración potencial.
RAI	1, 3, 6, 12 m	Anomalía firmada de precipitación respecto a la media histórica.
EDDI	1, 3, 6, 12 m	Anomalías en la demanda evaporativa atmosférica. Indicador de alerta temprana.
PDSI	Sin escala	Balance hídrico con oferta, demanda y humedad de suelo.

? Fuentes de datos disponibles

Fuente	Resolución	Proveedor	Clave
CHIRPS	0.05° (≈5.5 km)	UCSB/USGS	historical_chirps
IMERG	0.1° (≈11 km)	NASA GPM	historical_imerg
ERA5-Land	0.1° (≈11 km)	ECMWF	historical_era5_land
ERA5	0.25° (≈28 km)	ECMWF	historical_era5

? Categorías de severidad

Categoría	Umbral	Color
Humedad extrema	> +2.0	Azul oscuro
Humedad severa	+1.5 a +2.0	Azul
Humedad moderada	+1.0 a +1.5	Azul claro
Normal	−1.0 a +1.0	Blanco / Gris
Sequía moderada	−1.5 a −1.0	Amarillo
Sequía severa	−2.0 a −1.5	Naranja
Sequía extrema	< −2.0	Rojo oscuro

👉 Observación

Los índices hidrológicos (SDI, SRI, MFI, DDI, HDI) se calculan sobre las 29 estaciones del IDEAM y se visualizan como marcadores en el mapa, no como campo espacial continuo.

5.2. Predicción actual

La sección de predicción actual muestra el pronóstico probabilístico vigente para horizontes de 1 a 12 meses, leído del archivo `prediction_main.parquet`.

Cada predicción incluye:

- ✓ **Valor central:** valor predicho del índice por celda y horizonte.
- ✓ **Banda de incertidumbre:** rango intercuartílico Q1–Q3 representado como área sombreada en la gráfica.
- ✓ **Resumen IA:** interpretación en lenguaje natural generada automáticamente en español mediante la API Groq (modelo Llama 3.1-8b-instant).

? Anomalías de precipitación

Junto al mapa del índice, la predicción actual puede mostrar un mapa de **anomalías de precipitación**: el déficit o superávit de lluvia previsto, expresado en milímetros. La idea es traducir el SPI predicho —que es un número adimensional— a una cantidad física de lluvia, mucho más fácil de interpretar para la gestión del recurso.

Cómo se calcula. El SPI mide cuántas desviaciones estándar se aparta la precipitación de su valor normal. Para devolverlo a milímetros, SIPREH lo multiplica por la **desviación estándar histórica** de la precipitación acumulada de la celda:

$$A_c = \text{SPI}_c \times \sigma_c^P$$

donde SPI_c es el valor predicho del índice en la celda c y σ_c^P es la desviación estándar de la precipitación agregada de esa celda a lo largo del período climatológico de referencia. El resultado A_c está en milímetros: positivo indica más lluvia que lo normal, negativo indica déficit.

De dónde sale la climatología. La referencia σ_c^P se construye siempre a partir de la precipitación observada de **CHIRPS**, con estos pasos por celda:

1. Se acumula la precipitación mensual en una ventana móvil del tamaño de la escala (por ejemplo, 3 meses para una escala de 3).
2. Se toma ese acumulado para el período climatológico estándar **1991–2020** y se calcula su media y su desviación estándar.
3. La ventana se evalúa en el mes que corresponde al horizonte: el mes de emisión se desplaza tantos meses como el horizonte solicitado. Así, una emisión de enero con horizonte 12 compara contra los eneros del año siguiente, y el período 1991–2020 se desplaza a 1992–2021.

👉 Observación

El mapa de anomalías solo está disponible para el índice **SPI** y exige que la **escala de agregación coincida con el horizonte** de predicción (`escala = horizonte`). Es la condición que garantiza que el acumulado predicho y el acumulado climatológico se midan sobre la misma ventana de meses.

Tipo de anomalía	Significado	Color
Anomalía positiva fuerte	Lluvia muy por encima de lo normal	Azul intenso
Anomalía positiva	Lluvia sobre lo normal	Azul claro
Normal	Lluvia cercana a la climatología	Gris
Anomalía negativa	Lluvia bajo lo normal	Rojo claro
Anomalía negativa fuerte	Lluvia muy por debajo de lo normal	Rojo intenso

El mapa se actualiza al cambiar el horizonte en el panel lateral.

👉 Observación

Como las anomalías quedan en milímetros, son directamente comparables con registros de lluvia y con la demanda hídrica. Un déficit sostenido a lo largo de varios horizontes es una señal temprana de sequía acumulada.

👉 Observación

El resumen generado por IA es orientativo. Se recomienda contrastarlo con los valores numéricos del mapa y las gráficas de predicción.

5.3. Predicción histórica

La sección de predicción histórica permite comparar versiones de predicción emitidas en fechas anteriores. Cada archivo de predicción se identifica por su fecha de emisión (`issued_at`) y se archiva al cargar uno nuevo, sin sobrescribir el anterior.

Definición 5.1

issued_at: marca de tiempo que identifica la fecha de emisión de un archivo de predicción. Permite reconstruir la secuencia completa de pronósticos emitidos y comparar la estabilidad del modelo a lo largo del tiempo.

5.4. Administración

El módulo de administración centraliza la gestión operativa del sistema. Solo es accesible para usuarios con rol de administrador autenticados con JWT.

? Carga y actualización de archivos Parquet

La carga de datos es el flujo de trabajo más frecuente del administrador. A continuación se describe el proceso completo.

Convención de nombre de archivo. Idealmente, la resolución y la fuente de cada archivo se registran de forma explícita en sus metadatos. Como respaldo, el sistema también sabe inferirlas a partir del nombre del archivo, que debe contener una de estas palabras clave (sin distinción de mayúsculas):

Palabra clave en el nombre	Fuente detectada	Resolución
CHIRPS	CHIRPS	0.05° (≈5.5 km)
IMERG	IMERG	0.10° (≈11 km)
ERA5_LAND o era5land	ERA5-Land	0.10° (≈11 km)
ERA5	ERA5	0.25° (≈28 km)

Observación

ERA5 y ERA5-Land son fuentes distintas con resoluciones distintas (0.25° y 0.10° respectivamente), y el texto `era5_land` contiene la palabra `era5`. Por eso la detección comprueba siempre `ERA5_LAND` antes que `ERA5`: de lo contrario un archivo de ERA5-Land se confundiría con ERA5 y se dibujaría con celdas del tamaño equivocado. La misma regla de orden se aplica al inferir la fuente y la resolución, tanto en el backend como en el frontend.

Ejemplo 5.1

Nombres de archivo válidos:

- ✓ `grid_ERA5_LAND_2024.parquet` → ERA5-Land, 0.10°.
- ✓ `grid_ERA5_2024.parquet` → ERA5, 0.25°.
- ✓ `historical_CHIRPS_SPI.parquet` → CHIRPS, 0.05°.
- ✓ `prediction_main.parquet` → archivo de predicción (no requiere clave de fuente).

Observación

La resolución no es un dato cosmético: el frontend la usa para dibujar el tamaño de cada celda en el mapa. Si una fuente quedara registrada con la resolución de otra, las celdas se renderizarían demasiado grandes o demasiado pequeñas y se solaparían. Por eso conviene que cada archivo lleve su resolución correcta en los metadatos y que el nombre respete la convención anterior.

Pasos para cargar un nuevo archivo.

1. Ingresar al panel de administración con credenciales de administrador.
2. En la sección **Archivos**, hacer clic en **Cargar archivo**.
3. Seleccionar el archivo `.parquet` desde el sistema de archivos local.

- El sistema valida el formato y extrae automáticamente los metadatos (esquema de columnas, número de filas, rango de fechas, variables disponibles).
- Completar el formulario de configuración con los siguientes campos:

Campo	Descripción	Ejemplo
Dataset Key	Identificador del conjunto de datos al que pertenece el archivo.	historical_chirps
Role	Tipo de dato del archivo.	snapshot / prediction_monthly
Year-Month	Mes de referencia, formato YYYY-MM.	2024-06
Period Start	Fecha de inicio de la cobertura del archivo.	1990-01-01
Period End	Fecha de fin de la cobertura del archivo.	2024-12-31
Activate Now	Si se activa, el archivo queda disponible de inmediato.	Checkbox

- Hacer clic en **Guardar**. El archivo se sube a Cloudflare R2 y sus metadatos quedan registrados en la base de datos.
- Si no se marcó *Activate Now*, el dataset puede activarse posteriormente desde la sección **Datasets**.

Actualización de un archivo existente. Para reemplazar un archivo del mismo dataset con datos más recientes:

- Preparar el nuevo archivo Parquet con la misma convención de nombre y esquema de columnas.
- Seguir el mismo proceso de carga descrito arriba, seleccionando el mismo *Dataset Key*.
- El sistema registra el nuevo archivo como versión actualizada. El archivo anterior queda archivado (no eliminado) en R2.
- Activar el nuevo archivo desde la sección **Datasets** para que la interfaz lo use.

Observación

Los archivos de predicción (`prediction_main.parquet`) siguen el mismo proceso. Cada versión se identifica por su `issued_at`, lo que permite a los usuarios consultar predicciones históricas emitidas en fechas anteriores.

? Sincronización con Cloudflare R2

La sincronización bidireccional mantiene la consistencia entre la base de datos de metadatos y los archivos físicos en R2. Al sincronizar:

- ✓ Los archivos presentes en R2 pero no registrados en la base de datos local quedan registrados automáticamente.

- ✓ Los registros sin archivo correspondiente en R2 se marcan como no disponibles.

? **Gestión de usuarios**

- ✓ Creación de nuevos usuarios con rol estándar o administrador.
- ✓ Modificación de contraseña y datos de perfil.
- ✓ Desactivación de cuentas (no eliminación, para preservar la trazabilidad).

Capítulo 6 Funcionamiento del Sistema

Este capítulo describe los flujos de interacción típicos del usuario estándar y del administrador.

6.1. Flujograma de acceso del usuario

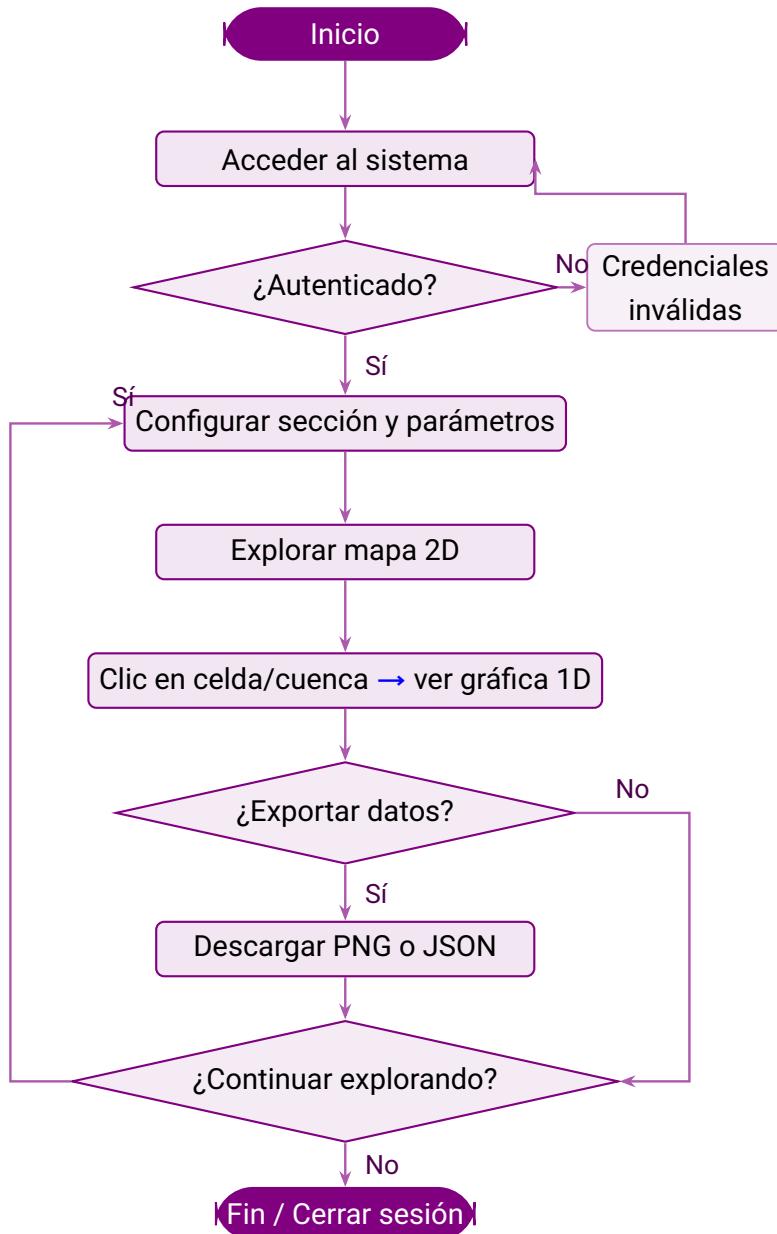


Figura 6.1. Flujo de acceso y uso del sistema para un usuario estándar.

6.2. Flujograma de acceso del administrador

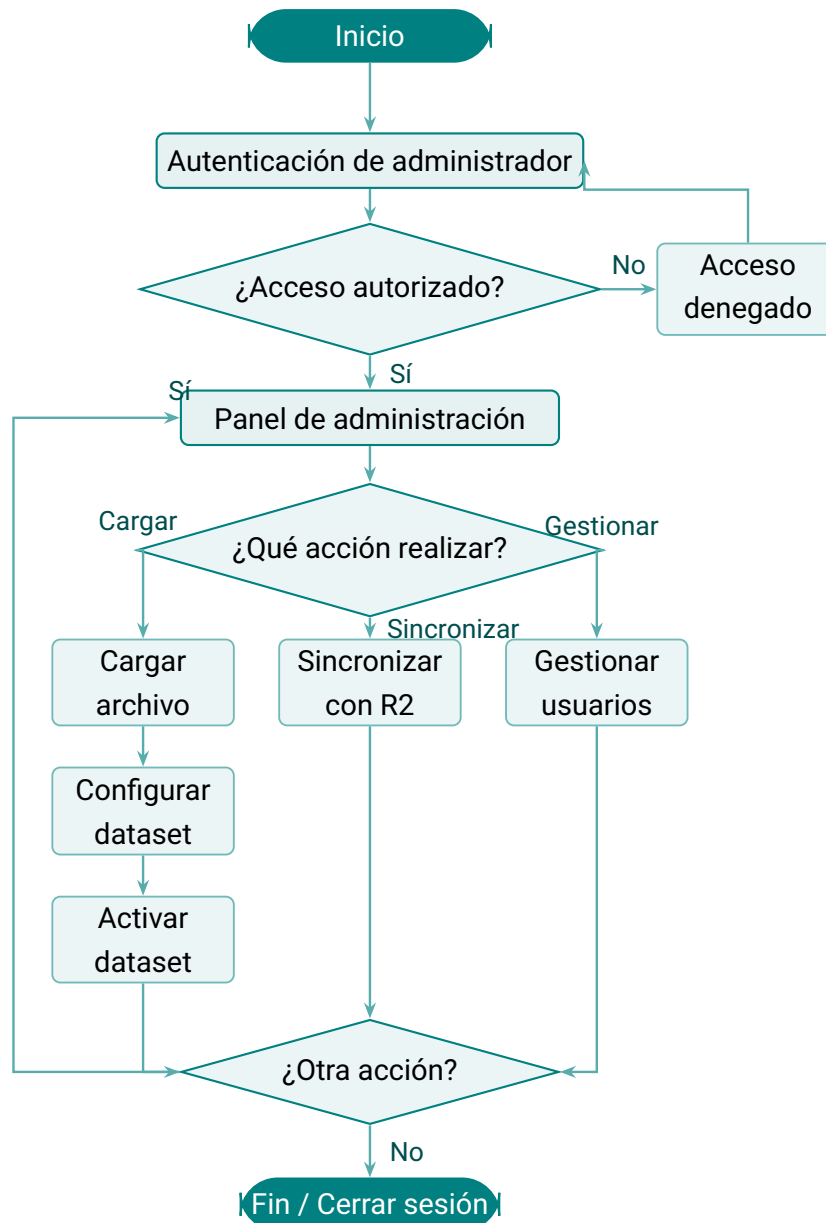


Figura 6.2. Flujo de acceso y operación para un administrador del sistema.

Capítulo 7 Estructura del Proyecto

Este capítulo describe la organización de directorios del proyecto SIPREH en el frontend y el backend.

7.1. Estructura del frontend

El frontend es una aplicación Next.js 14 que sigue las convenciones del *App Router*:

```
drought-frontend/
+-- public/
|   +-- watersheds/           # GeoJSON de cuencas hidrografica
|   +-- icons/                # Iconos de marcadores
+-- src/
|   +-- app/
|   |   +-- layout.tsx        # Layout raiz
|   |   +-- page.js           # Pagina principal (mapa + sidebar)
|   |   +-- login/page.tsx    # Pagina de autenticacion
|   |   +-- admin/page.tsx    # Panel de administracion
|   +-- components/
|   |   +-- map/              # Componentes del mapa Leaflet
|   |   +-- charts/           # Graficas uPlot y Canvas 2D
|   |   +-- sidebar/          # Paneles de historico y prediccion
|   |   +-- admin/
|   |       +-- dashboard/
|   |           +-- FilesSection.js  # Carga de archivos Parquet
|   |           +-- DatasetConfig.js # Configuracion de datasets
|   |           +-- UserManager.js   # Gestion de usuarios
|   |           +-- SyncStatus.js    # Estado de sincronizacion R2
|   +-- utils/
|   |   +-- exporters.js       # Exportacion PNG y JSON
|   |   +-- lttb.js            # Algoritmo de diezmado LTTB
|   |   +-- colorScale.js      # Paleta de colores de severidad
|   +-- lib/
|       +-- api.js             # Cliente HTTP hacia el backend
|       +-- auth.js            # Gestion de tokens JWT
+-- .env.local                 # NEXT_PUBLIC_API_URL
+-- next.config.js
+-- package.json
```

Observación

La función de exportación (`src/utils/exporters.js`) genera imágenes PNG de 1800×1100 px con encabezado institucional y metadatos, y archivos JSON con los datos de la consulta activa

(columnas, fuente, resolución, sistema de referencia WGS84).

7.2. Estructura del backend

El backend es una aplicación FastAPI organizada en capas funcionales:

```
drought-backend/
+-- app/
|   +-- main.py           # Punto de entrada FastAPI
|   +-- config.py         # Variables de entorno y settings
|   +-- database.py        # Inicializacion SQLAlchemy
|   +-- models/           # Modelos ORM (usuario, dataset, archivo)
|   +-- api/
|       +-- v1/
|           +-- api.py     # Registro de routers
|           +-- endpoints/
|               +-- auth.py      # /auth: login, refresh
|               +-- parquet.py   # /parquet: carga de archivos
|               +-- historical.py # /historical: 1D, 2D, catalogo
|               +-- prediction.py # /prediction: actual e historica
|               +-- hydro.py     # /hydro: estaciones IDEAM
|               +-- drought.py   # /drought: analisis de sequia
|               +-- dashboard.py # /dashboard: resumen del mapa
|               +-- admin_files.py # /admin/files: CRUD archivos
|               +-- admin_datasets.py # /admin/datasets: ciclo datasets
|               +-- admin_users.py # /admin/users: gestion usuarios
|               +-- admin_cloud.py # /admin/cloud: integracion R2
|               +-- admin_tiered.py # /admin/tiered: almac. escalonado
|               +-- admin_utils.py # Utilidades compartidas de admin
|   +-- services/
|       +-- historical_data_service.py
|       +-- prediction_data_service.py
|       +-- hydro_data_service.py
|       +-- drought_analysis.py
|       +-- ai_summary_service.py
|       +-- cache.py
|       +-- cloud_storage.py
|       +-- parquet_processor.py
|       +-- tiered_storage.py
|   +-- schemas/           # Esquemas Pydantic de request/response
+-- .env                   # Variables de entorno (BD, R2, JWT, Redis)
+-- requirements.txt
```

```
+-- Dockerfile
+-- docker-compose.yml
```

Ejemplo 7.1

Flujo de una consulta 1D de serie de tiempo:

1. El router `historical.py` recibe GET `/api/v1/historical/timeseries` con parámetros `lat`, `lon`, `index` y `scale`.
2. `historical_data_service.py` verifica el caché (`cache.py`).
3. Si no hay caché, DuckDB consulta el archivo Parquet en disco.
4. El resultado se almacena en caché y se devuelve como JSON.

Capítulo 8 Despliegues

SIPREH se despliega siguiendo la misma separación que define su arquitectura: el frontend y el backend viven en plataformas distintas y se comunican únicamente a través de la API HTTP. En la práctica esto significa dos despliegues independientes que pueden actualizarse por separado:

- ✓ El **frontend** se publica como sitio estático en **GitHub Pages**.
- ✓ El **backend** corre como servicio web en **Railway**, acompañado de la base de datos relacional y del bucket de objetos en Cloudflare R2.

Este capítulo describe cómo funciona cada despliegue en producción y qué se necesita para reproducirlo.

8.1. Despliegue frontend

El frontend es una aplicación **Next.js 16** que, para producción, se compila como **exportación estática**. En lugar de un servidor Node.js permanente, el resultado es una carpeta `out/` con HTML, CSS y JavaScript que cualquier hosting de archivos estáticos puede servir. GitHub Pages cumple ese rol sin costo adicional.

? Cómo se publica

El despliegue está automatizado con GitHub Actions. El flujo vive en `.github/workflows/deploy.yml` y se dispara con cada *push* a la rama `main`. No hay que ejecutar nada a mano: basta con fusionar a `main`.

El flujo realiza dos trabajos encadenados:

1. **Build.** Instala las dependencias del frontend con `npm ci`, compila la exportación estática con `npm run build` y sube la carpeta `out/` como artefacto de Pages.
2. **Deploy.** Publica ese artefacto en el entorno `github-pages`, que queda disponible en la URL del proyecto.

? La pieza clave: a qué backend apunta el sitio

Como el sitio es estático, la dirección del backend se “hornea” en el código **durante el build**, no en tiempo de ejecución. Por eso la variable que la define debe estar disponible en el momento de compilar:

```
NEXT_PUBLIC_API_URL=https://<tu-backend>.up.railway.app/api/v1
```

En el flujo de GitHub Actions, este valor se inyecta desde una *variable de repositorio* llamada `RENDER_BACKEND_URL`. Para cambiar de backend (por ejemplo, al migrar de entorno), se edita esa variable en `Settings` → `Secrets and variables` → `Actions` del repositorio y se vuelve a desplegar.

 Observación

Como la URL del backend queda fijada en el build, un cambio en `RENDER_BACKEND_URL` solo surte efecto tras un nuevo despliegue. Volver a ejecutar el flujo (o hacer un *push* a `main`) regenera el sitio con la nueva dirección.

 El detalle del `basePath`

Un sitio de GitHub Pages de proyecto no se sirve en la raíz del dominio, sino bajo una subruta con el nombre del repositorio (por ejemplo, `https://<organización>.github.io/SIPREH/`). Si la aplicación asumiera que vive en la raíz, las imágenes y los enlaces internos se romperían.

Para evitarlo, `next.config.mjs` detecta la variable `GITHUB_ACTIONS` (que GitHub define automáticamente en cada ejecución) y, solo en ese caso, configura `basePath` y `assetPrefix` con el nombre del repositorio. En desarrollo local esa variable no existe, así que la aplicación funciona en `localhost:3000` sin prefijo y sin cambios.

Ejemplo 8.1

Comportamiento según el entorno, controlado por una sola condición en `next.config.mjs`:

```
# Local (GITHUB_ACTIONS no definida)
http://localhost:3000/

# GitHub Pages (GITHUB_ACTIONS=true)
https://<organización>.github.io/SIPREH/
```

8.2. Despliegue backend

El backend es una aplicación **FastAPI + Uvicorn** que se despliega en **Railway**. Railway construye el servicio directamente desde el repositorio y lo mantiene en línea, exponiéndolo en una URL pública del tipo `https://<servicio>.up.railway.app`.

 Modo producción

El punto de entrada `run.py` decide automáticamente si está en producción. Cuando detecta la variable `RAILWAY_ENVIRONMENT` (que Railway inyecta sola) o `RENDER`, o cuando `DEBUG` no está definida, desactiva la recarga automática de código y escucha en el puerto que la plataforma indique mediante la variable `PORT`. En desarrollo local, en cambio, la recarga queda activa para agilizar el trabajo.

 Observación

No es necesario configurar el puerto manualmente en Railway: la plataforma asigna `PORT` y `run.py` la respeta. Tampoco se necesita un `Procfile` ni un servidor de procesos externo; Uvicorn es el servidor de la aplicación.

? Variables de entorno

Toda la configuración se carga desde variables de entorno (en local, desde un archivo `.env`; en Railway, desde el panel del servicio). La gran mayoría son opcionales y tienen valores por defecto razonables: el backend arranca aunque falten R2, Redis o Groq, degradándose con elegancia. Las relevantes para un despliegue completo son:

```
# Base de datos relacional (metadatos, usuarios, registro de archivos)
DATABASE_URL=postgres://user:password@host:5432/sipreh
```

```
# Seguridad
SECRET_KEY=<clave_secreta_larga_y_aleatoria>
BACKEND_CORS_ORIGINS=["https://<organización>.github.io"]
```

```
# Almacenamiento de objetos (Cloudflare R2, compatible con S3)
CLOUD_STORAGE_PROVIDER=cloudflare-r2
CLOUD_STORAGE_ENDPOINT=https://<account_id>.r2.cloudflarestorage.com
CLOUD_STORAGE_ACCOUNT_ID=<account_id>
CLOUD_STORAGE_ACCESS_KEY=<clave_de_acceso>
CLOUD_STORAGE_SECRET_KEY=<clave_secreta>
CLOUD_STORAGE_BUCKET=drought-data
CLOUD_STORAGE_REGION=auto
```

```
# Caché de resultados (opcional pero muy recomendable)
REDIS_URL=redis://<host>:6379/0
```

```
# Resúmenes con IA (opcional)
GROQ_API_KEY=<clave_groq>
```

```
# Usuario administrador inicial
ADMIN_EMAIL=admin@sipreh.co
ADMIN_PASSWORD=<contraseña_inicial>
```

Observación

`BACKEND_CORS_ORIGINS` debe incluir el dominio de GitHub Pages desde el que el frontend hace las peticiones; de lo contrario el navegador bloqueará las respuestas por política de CORS. Es el ajuste que más suele olvidarse al conectar ambos despliegues por primera vez.

? Inicialización de la base de datos

La primera vez que se prepara un entorno, se ejecuta `python init_db.py`. Este script crea las tablas, da de alta el usuario administrador definido por `ADMIN_EMAIL/ADMIN_PASSWORD` y normaliza metadatos antiguos. El esquema se genera directamente con SQLAlchemy (`create_all`); el proyecto no usa migraciones Alembic, de modo que para un primer despliegue basta con este paso.

? Verificación

Una vez en línea, la API publica su documentación interactiva, útil para comprobar que el servicio responde:

- ✓ `/docs` — Swagger UI (interfaz de prueba interactiva).
- ✓ `/redoc` — ReDoc (documentación de referencia).
- ✓ `/api/v1/historical/catalog` — catálogo de datos disponibles (requiere autenticación).

8.3. Almacenamiento y base de datos

SIPREH separa deliberadamente dos tipos de almacenamiento: la base de datos relacional, que guarda solo metadatos, y el bucket de objetos, que guarda los datos científicos pesados. Cada uno se despliega de forma distinta.

? Base de datos relacional

Contiene los metadatos del sistema: usuarios, el registro de archivos Parquet y las relaciones geográficas entre celdas y zonas. Es liviana, y por eso su despliegue es sencillo:

- ✓ **Desarrollo:** SQLite, sin instalación. El archivo se crea solo al iniciar el backend, en la ruta de `DATABASE_URL`.
- ✓ **Producción:** PostgreSQL, idealmente como servicio gestionado. Railway ofrece PostgreSQL como complemento del mismo proyecto, lo que mantiene backend y base de datos juntos.

Ejemplo 8.2

Una misma variable cubre ambos entornos:

```
# SQLite (desarrollo local)
DATABASE_URL=sqlite:///./droughtmonitor.db

# PostgreSQL (producción, p. ej. en Railway)
DATABASE_URL=postgresql://sipreh_user:password@host:5432/sipreh
```

? Almacenamiento de objetos Cloudflare R2

El bucket R2 guarda todos los archivos Parquet con los datos hidrometeorológicos. La configuración inicial es:

1. Crear una cuenta en [Cloudflare](#) y activar R2 desde el panel.
2. Crear un bucket con el nombre que usará `CLOUD_STORAGE_BUCKET` (por ejemplo, `drought-data`).
3. Generar un token de API de R2 con permisos de lectura y escritura sobre ese bucket.
4. Llevar las credenciales a las variables `CLOUD_STORAGE_*` del backend.
5. Probar la conexión ejecutando la sincronización desde el panel de administración de SIPREH.

👉 Observación

A diferencia de Amazon S3, Cloudflare R2 no cobra por egreso (egress). Esto encaja con el patrón de acceso de SIPREH, donde los archivos Parquet (de 100 MB a varios GB) se descargan desde el bucket en el primer acceso o tras un nuevo despliegue.

Definición 8.1

Egress (en almacenamiento en la nube): transferencia de datos desde el proveedor hacia el exterior. Servicios como Amazon S3 o Google Cloud Storage cobran por GB de egreso, un costo que puede crecer rápido en aplicaciones que sirven archivos de datos grandes con frecuencia.

? Resumen de infraestructura

Componente	Entorno dev	Entorno prod	Plataforma
Frontend	localhost:3000	Sitio estático	GitHub Pages
Backend	localhost:8000	Servicio web FastAPI	Railway
Base relacional	SQLite	PostgreSQL 15+	Railway
Caché	localhost:6379	Redis gestionado	Railway / externo
Datos Parquet	Caché en disco local	Bucket de objetos	Cloudflare R2